

PRX

Informe unificado del proyecto Desarrollo y revisión del Sprint 2

Aplicación web para organizar repositorios, notas y recursos compartidos.

Documento	Informe técnico y metodológico
Versión	Entrega documental - 09 de octubre de 2026
Tecnologías	Angular, NestJS, Prisma, MySQL y Fabric.js
Alcance principal	HU04: multimedia; HU05: edición de imágenes; HU06: visualización de notas.

Resumen ejecutivo

PRX permite reunir información y recursos en repositorios. El Sprint 2 amplió el trabajo con notas mediante archivos adjuntos, edición de imágenes y una vista de detalle que integra contenido, enlaces y tareas. También se mejoró la presentación del listado para localizar notas mediante tarjetas con portada, sin que cada adjunto aumente su altura.

Este informe unifica funcionalidades, organización del trabajo, modelo de negocio, arquitectura, pruebas y rutas del código. Diferencia lo observado en el proyecto de las propuestas metodológicas y de los aspectos pendientes de validación.

Contenido

1. Contexto, objetivos y alcance	2
2. Desarrollo del Sprint 2	3
3. Arquitectura, patrones y SOLID	4
4. Organización del trabajo: Kanban y XP	5
5. Business Model Canvas de PRX	6
6. Pruebas, integración y estado de entrega	7
7. Integración en el código de PRX	8
8. Anexo: mapa de pantallas y código	9
9. Frontend: rutas y código	10-12
10. Backend: rutas y código	13-15
11. Validación y configuración de entrega	16

1. Contexto y objetivos

El problema que atiende PRX y el aporte concreto del Sprint 2.

Necesidad del proyecto

Al trabajar con recursos de un mismo tema, los textos, imágenes, documentos y enlaces pueden quedar repartidos entre distintas herramientas. Esto dificulta encontrarlos y relacionarlos. PRX plantea un espacio común: el repositorio agrupa los recursos y la nota reúne contenido y adjuntos vinculados a una actividad o tema.

Objetivo general

Facilitar la organización y consulta de información compartida mediante una aplicación web de repositorios y notas, incorporando herramientas para preparar imágenes y consultar recursos desde el mismo flujo de trabajo.

Objetivos específicos del Sprint 2

1. Permitir adjuntar documentos e imágenes a las notas con validaciones de cantidad y tamaño.
2. Editar imágenes antes de guardarlas, incluyendo anotaciones, transformaciones y filtros.
3. Consultar una nota con sus enlaces, tareas y recursos asociados.
4. Mejorar la navegación y la identificación visual de notas mediante tarjetas compactas.
5. Revisar el guardado y la recuperación de información mediante pruebas y recorridos manuales.

Contexto funcional de PRX

El proyecto incluye registro e inicio de sesión, exploración de repositorios públicos, repositorios propios y compartidos, roles de participación y notas. Estos elementos dan contexto al Sprint 2. La creación de cuentas y los ajustes de acceso se trataron como trabajo adicional; no se contabilizan dentro de las tres historias de este Sprint.

Usuarios y recorrido principal

Una persona inicia sesión, entra en un repositorio y abre la sección Notas. Desde allí puede crear una nota, escribir contenido, adjuntar recursos y editar una imagen. Después de guardar, la nota aparece en el listado. Al abrirla, el usuario consulta su contenido y los recursos disponibles, según sus permisos.

Alcance y límites de esta entrega

El informe documenta el módulo de notas y sus conexiones con la aplicación. No presenta como desarrolladas funciones futuras, ni afirma que el proyecto tiene un modelo comercial aprobado. Se comprobaron compilación, pruebas automatizadas y operaciones HTTP reales al 09/10/2026. La representación visual de PDF depende del visor del navegador; existen apertura en pestaña y descarga como alternativas.

2. Desarrollo del Sprint 2

Historias de usuario, comportamiento implementado y mejoras de interfaz.

Historia / esfuerzo	Trabajo realizado
HU04 - Multimedia 8 puntos	Carga de documentos e imágenes, miniaturas, eliminación de adjuntos y relación con la nota. Validación conjunta de hasta 5 archivos y 50 MiB por archivo.
HU05 - Editor 8 puntos	Editor integrado con pincel, marcador, texto, formas, flechas, recorte, rotación, reflejo, filtros, dimensiones, zoom, deshacer y rehacer. Exportación del resultado en PNG.
HU06 - Ver nota 5 puntos	Detalle con título, autor, contenido, enlaces, tareas y recursos. Rediseño del detalle y de las tarjetas del listado.

Esfuerzo planificado: 21 puntos en las tres historias. Los puntos representan una estimación relativa de esfuerzo, no horas ni un porcentaje de avance.

Adjuntos y guardado

El formulario distingue documentos e imágenes, pero aplica un límite conjunto. El backend coordina el almacenamiento y la persistencia de la nota. Cuando falla el guardado, contempla la limpieza de los archivos recién subidos; al reemplazar o eliminar adjuntos, la eliminación de recursos anteriores se coordina con la confirmación de los datos.

Editor de imágenes

Se corrigió la posición de la imagen para que el lienzo muestre el contenido completo. La entrada de texto se ajustó para poder escribir dentro del diálogo. El usuario puede cambiar fuente, tamaño, color, estilo y alineación; además, dispone de filtros y ajustes de tamaño conservando la proporción. El zoom ayuda a trabajar con distintas dimensiones de pantalla.

Límite del editor: al guardar se genera un PNG con el resultado visual. Las anotaciones quedan integradas en la imagen; no se recuperan como capas editables al volver a abrirla.

Listado y detalle de notas

El listado sustituyó las filas muy anchas por tarjetas con una portada y un resumen de adjuntos. Así, una nota con varios archivos conserva una altura estable. En Ver nota, el contenido, los enlaces, las tareas y los archivos se organizan en secciones; las tareas incluyen un indicador de progreso y las imágenes se presentan en galería.

Recorrido para demostrar el Sprint

Abrir **Mis repositorios / un repositorio / Notas**. Crear una nota con título y contenido, adjuntar una imagen, pulsar Editar, escribir una anotación y guardar. Confirmar la tarjeta en el listado y abrir Ver nota para revisar contenido, tareas, enlaces y adjuntos. Los PDF se recuperan como blob y pueden abrirse en el diálogo, en pestaña o descargarse. La visualización embebida requiere un navegador con visor PDF.

3. Arquitectura y calidad

Cómo se organiza el código y qué patrones se identificaron.

Separación por capas

El frontend Angular presenta los formularios y coordina el estado de la interfaz. El backend NestJS recibe las peticiones, ejecuta operaciones de aplicación y delega la persistencia a repositorios. Prisma conecta los datos con MySQL y el servicio de almacenamiento gestiona los archivos. Fabric.js proporciona el lienzo y los objetos del editor de imágenes.

Capa	Responsabilidad dentro de PRX
Presentación	Pantallas y componentes Angular; controlador HTTP de notas en NestJS.
Aplicación	Fachada de notas, comandos de creación y actualización, consultas y servicio multimedia.
Dominio	Contratos de repositorios y entidades de notas.
Infraestructura	API y estado en frontend; repositorio Prisma, mapeadores y almacenamiento en backend.

Patrones presentes

Repository: NoteRepository define el contrato y PrismaNoteRepository implementa el acceso a datos.

CQRS: NotesController delega las operaciones mediante CommandBus y QueryBus.

Facade: NoteFacade coordina las llamadas a la API y el estado de notas.

Mapper: los mapeadores convierten datos de persistencia en entidades o respuestas.

Inyección de dependencias: NestJS registra y proporciona las implementaciones requeridas.

Revisión de SOLID

S - Responsabilidad única: hay separación entre controladores, operaciones y persistencia. El detalle de nota todavía reúne edición, tareas, enlaces y visualización de archivos.

O - Abierto/cerrado: los contratos facilitan cambios de persistencia. Añadir filtros de imagen exige modificar la lógica existente; una estrategia de filtros sería una mejora.

L - Sustitución: no se identificó una ruptura evidente del contrato del repositorio, pero falta una verificación sistemática de comportamiento.

I - Segregación: existen contratos separados, aunque algunos puertos CRUD pueden exigir operaciones que ciertos consumidores no necesitan.

D - Inversión de dependencias: se usan contratos en persistencia; la fachada depende de NoteApi concreto y el servicio multimedia del almacenamiento Tigris concreto.

Resultado: el proyecto incorpora patrones y aplica SOLID de forma parcial. Se propone extraer un visor de archivos y definir contratos para la API y el almacenamiento. Estas mejoras no se presentan como implementadas.

4. Organización del trabajo

Kanban y XP vinculados a las tareas reales del Sprint 2.

Kanban y división del trabajo en PRX

Kanban permite visualizar y organizar tareas concretas. Para explicar el trabajo del Sprint 2, se agrupan las actividades realizadas en tres bloques: archivos multimedia, editor de imágenes y visualización de notas. Esta presentación relaciona cada historia con sus resultados, sin reproducir un tablero de estados ni atribuir al equipo un tablero formal que no se verificó.

Bloque de trabajo	Actividades realizadas en PRX
HU04 - Multimedia	Se incorporó la selección de documentos e imágenes, las miniaturas, los límites de carga y la asociación de adjuntos con la nota.
HU05 - Editor de imágenes	Se incorporaron herramientas de anotación y transformación. Se corrigieron la posición inicial de la imagen y la escritura de texto en el diálogo.
HU06 - Visualización de notas	Se organizaron contenido, enlaces, tareas y recursos en el detalle. Se mejoraron las tarjetas y el resumen de adjuntos del listado.

Secuencia de trabajo realizada

Se trabajó sobre el formulario y la selección de recursos, la coordinación del guardado en el backend, las herramientas del editor y la presentación de las notas. Los problemas detectados durante los recorridos de uso dieron lugar a ajustes concretos: mostrar la imagen completa, permitir escribir texto y evitar que las tarjetas crezcan con cada adjunto. El apartado de pruebas describe por separado el alcance de la validación.

XP para la calidad del código

En este proyecto, XP se relaciona con realizar cambios pequeños, comprobarlos y mejorar la estructura cuando aparecen responsabilidades mezcladas. Las utilidades del editor permiten probar filtros y dimensiones por separado. Los servicios y handlers de adjuntos permiten comprobar casos de guardado y limpieza cuando ocurre un error.

Práctica de XP	Relación con el proyecto
Pruebas continuas	Mantener pruebas de filtros, dimensiones y coordinación del guardado de archivos.
Refactorización	Extraer operaciones del lienzo y separar responsabilidades de la pantalla de detalle.
Integración frecuente	Conectar y verificar cambios pequeños entre pantalla, API y persistencia.
Programación en parejas	Propuesta útil para revisar permisos y fallos del editor; no se confirmó su uso.

Estado metodológico

Se identificaron pruebas, separación de utilidades y ajustes de estructura. No se confirmó un proceso formal de XP, programación en parejas, TDD estricto ni una canalización de integración continua. Estas prácticas se explican como guía de trabajo para PRX, sin atribuir al equipo actividades no verificadas.

5. Modelo de negocio de PRX

Business Model Canvas elaborado como propuesta del proyecto.

El Canvas relaciona el valor de PRX con sus usuarios, operación y sostenibilidad. La siguiente propuesta se basa en las funciones observadas de repositorios y notas. No constituye un modelo comercial aprobado ni confirma ingresos actuales.

Bloque	Aplicación a PRX
1. Segmentos de clientes	Personas y equipos que necesitan organizar y compartir información y recursos por tema o actividad.
2. Propuesta de valor	Reunir repositorios, notas, documentos, imágenes, enlaces y tareas en un espacio web, con edición básica de imágenes dentro del flujo.
3. Canales	La aplicación web es el canal observado. La documentación y las demostraciones del proyecto pueden apoyar su difusión.
4. Relaciones con clientes	Uso mediante una cuenta y espacios compartidos. Se propone ayuda contextual y atención de incidencias para facilitar la adopción.
5. Fuentes de ingresos	Por definir. No se verificó un sistema de cobros. Antes de proponer suscripciones se debe validar la necesidad y la disposición de los usuarios.
6. Recursos clave	Código Angular y NestJS, base de datos MySQL, almacenamiento de archivos, infraestructura y equipo de desarrollo.
7. Actividades clave	Desarrollar repositorios y notas, probar operaciones y permisos, mantener el almacenamiento y resolver problemas de uso.
8. Socios clave	Proveedores de alojamiento y almacenamiento y colaboradores del proyecto. Las alianzas concretas deben definirse.
9. Estructura de costos	Alojamiento, base de datos, capacidad y transferencia de archivos, desarrollo, mantenimiento y soporte. No se verificaron importes.

Cómo aporta el Sprint 2 a la propuesta de valor

La carga de recursos evita separar la nota de sus evidencias. El editor reduce pasos al preparar imágenes dentro de PRX. La vista de detalle y las tarjetas con portada ayudan a consultar y reconocer el contenido. Estos aportes deben validarse con usuarios, especialmente al trabajar con documentos y varios adjuntos.

Decisiones por validar

Comprobar qué usuarios requieren este espacio, qué límites de almacenamiento son adecuados y cuánto cuesta mantenerlo. La sostenibilidad comercial no se deduce únicamente del código; requiere contrastar necesidades, uso y costos reales.

6. Pruebas y estado de entrega

Diferencia entre comprobar piezas aisladas y el recorrido completo.

Pruebas unitarias en PRX

Se localizaron pruebas para las utilidades del editor, el detalle de nota, el servicio multimedia y los handlers de archivos. Comprueban operaciones concretas, como aplicar un filtro, validar dimensiones o coordinar la limpieza de subidas cuando falla la persistencia. Las dependencias simuladas permiten aislar estas operaciones, pero no sustituyen una prueba de integración real.

Pruebas de integración del proyecto

El recorrido relevante conecta **Angular / API NestJS / Prisma y MySQL / almacenamiento / Ver nota**. Debe comprobarse que una nota guardada conserva sus archivos y que estos se recuperan y abren con los permisos adecuados. Se ejecutó una comprobación HTTP contra NestJS, MySQL y el almacenamiento local: alta de repositorio, nota con PDF y PNG, descarga con igualdad de bytes, actualización de tareas y enlaces, conservación de adjuntos y rechazo por permisos. Los datos temporales se retiraron.

Tipo de integración	Aplicación o alternativa dentro de PRX
Incremental	Conectar primero notas, después adjuntos y luego el editor, comprobando cada paso. Es la opción recomendada para seguir desarrollando el módulo.
Big Bang	Integrar interfaz, API, datos y archivos al final. Dificultaría localizar el origen de fallos del módulo.
Top-Down	Empezar por las pantallas y simular respuestas de la API mientras se construyen operaciones inferiores.
Bottom-Up	Comprobar repositorios y almacenamiento antes de conectar servicios, controladores y pantallas.

Top-Down y Bottom-Up también pueden realizarse de forma incremental. Son alternativas de organización; no se afirma que el equipo haya utilizado todas.

Criterios de aceptación para cerrar el Sprint

Notas: crear, recuperar y actualizar contenido sin pérdida de datos.

Adjuntos: aplicar límites, mantener la asociación, recuperar recursos y manejar fallos sin dejar subidas inconsistentes.

Editor: mostrar la imagen completa, escribir texto, transformar y guardar el resultado.

Interfaz: conservar tarjetas compactas y permitir navegar entre listado y detalle.

Documentos: confirmar apertura y descarga de PDF y otros formatos admitidos.

Resultados y límites de la revisión

Compilaciones frontend y backend correctas; 22 pruebas frontend y 21 backend aprobadas. Las operaciones HTTP de notas, documentos y permisos dieron resultado correcto. El PNG exportado no conserva capas. SOLID se aplica parcialmente. La vista PDF usa el visor nativo del navegador; no se certificó su representación en todos los navegadores ni la conexión al servicio Tigris de producción.

7. Integración en el código de PRX

Puntos reales donde se conectan los módulos del Sprint 2.

Recorrido: formulario Angular / NoteFacade / NoteApi / HTTP multipart / NotesController / CreateNoteHandler / almacenamiento y Prisma. Las rutas de esta página parten de la raíz indicada en el anexo; las páginas siguientes incluyen sus rutas completas.

1. Frontend: empaquetar contenido y archivos

prx-frontend/src/app/features/notes/infrastructure/api/note.api.ts

Fragmento real, líneas 43-49. FormData integra los datos de la nota con los adjuntos antes del POST.

```
43 formData.append('title', data.title);
44 formData.append('content', data.content);
45 formData.append('links', JSON.stringify(data.links));
46
47 files.forEach((file) => {
48   formData.append('files', file, file.name);
49 });
```

2. Backend: delegar la solicitud HTTP

prx-backend/src/modules/notes/presentation/controllers/notes.controller.ts

Fragmento real, líneas 81-84. El controlador entrega datos, archivos y usuario al comando de creación.

```
81 ) {
82   return this.commandBus.execute(
83     new CreateNoteCommand(repositoryId, createNoteRequestDto, userId, files),
84   );
```

3. Aplicación: almacenamiento y persistencia

prx-backend/src/modules/notes/application/commands/create-note/create-note.handler.ts

Fragmento real, líneas 84-91. Coordina subida, guardado y limpieza si falla la persistencia.

```
84 const uploaded = await this.mediaStorage.upload(command.files ?? [], command.repositoryId, comm
    and.createdBy);
85 let created: NoteEntity;
86 try {
87   created = await this.noteRepository.createWithFiles(note, uploaded);
88 } catch (error) {
89   await this.mediaStorage.remove(uploaded.map((file) => file.storagePath));
90   throw error;
91 }
```

4. Datos: transacción y relación de adjuntos

prx-backend/src/modules/notes/infrastructure/persistence/prisma-note.repository.ts

Fragmento real, líneas 166-169. La transacción contiene el alta de la nota y sus noteFiles.

```
166 tasks: entity.tasks as unknown as Prisma.InputJsonValue,
167 links: entity.links as unknown as Prisma.InputJsonValue,
168 createdBy: entity.createdBy,
169 noteFiles: { create: files },
```

Relación con los tipos de integración: estos son puntos técnicos del recorrido; el código no prueba si el equipo siguió Top-Down, Bottom-Up o Big Bang. Se recomienda validarlos incrementalmente: API y datos, adjuntos y después el editor. La descarga PDF se comprobó mediante comparación de bytes; la representación embebida depende del visor nativo del navegador.

8. Anexo: mapa del código

Rutas para localizar las pantallas y seguir el recorrido del módulo.

Raíz real: D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/

Las rutas siguientes se añaden a esta raíz. En VS Code, Ctrl + P permite buscar cada archivo por su nombre.

Pantallas de notas

/notes/repositories/:repositoryId

/notes/repositories/:repositoryId/create

/notes/repositories/:repositoryId/:noteId

Corresponden a listado, creación y detalle. El editor se abre como componente dentro de la página.

Base frontend: prx-frontend/src/app/features/notes/

Ruta relativa a la base frontend	Qué revisar
presentation/pages/notes-page/ notes-page.component.ts	Listado, tarjetas y contador de adjuntos.
presentation/pages/create-note-page/ create-note-page.component.ts	Formulario, selección y envío de archivos.
presentation/pages/note-detail-page/ note-detail-page.component.ts	Detalle, tareas, enlaces y apertura de recursos.
presentation/components/note-image-editor/ note-image-editor.component.ts	Lienzo y herramientas. image-editor.utils.ts contiene utilidades.
application/facades/note.facade.ts infrastructure/api/note.api.ts	Coordinación de estado y comunicación HTTP.

Los .html y .scss junto a cada componente definen la estructura y el estilo. presentation/notes.routes.ts registra las URLs. Las pruebas frontend están junto a las utilidades y al componente de detalle.

Base backend: prx-backend/src/modules/notes/

Ruta relativa a la base backend	Qué revisar
presentation/controllers/notes.controller.ts	Endpoints HTTP y delegación CQRS.
application/commands/create-note/ create-note.handler.ts	Creación. La actualización está en update-note/update-note.handler.ts.
application/services/note-media-storage.service.ts	Validación y coordinación de adjuntos.
domain/repositories/note.repository.ts infrastructure/persistence/prisma-note.repository.ts	Contrato e implementación de persistencia.

Otras referencias desde la raíz: prx-backend/prisma/schema.prisma;

prx-backend/src/shared/infrastructure/storage/tigris-storage.service.ts. Pruebas backend:

application/services/note-media-storage.service.spec.ts y application/commands/note-media-handlers.spec.ts, bajo la base de notas.

API (localhost:3000): GET/POST /prx/notes/repository/:repositoryId; GET/PUT/DELETE /prx/notes/:id; GET /prx/notes/files/:id/download.

Base documental: revisión del código y recorridos registrados del proyecto PRX al 09/10/2026. El documento describe el módulo, sus límites y propuestas; no equivale a un acta de aceptación final.

9. Frontend: envío de la nota

Rutas completas y fragmentos reales del recorrido de creación.

Los bloques oscuros son extractos del código leído en el proyecto. Se omiten las partes externas al intervalo indicado. Los números corresponden al archivo original al 09/10/2026; pueden cambiar si se edita el código.

Formulario: llamada a la fachada

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/pages/create-note-page/create-note-page.component.ts

Fragmento real, líneas 291-301. createNote envía el repositorio, el contenido y los archivos; al completar, vuelve al listado.

```
291 this.noteFacade
292   .create(repositoryId, request, files)
293   .pipe(
294     finalize(() => {
295       this.submitting.set(false);
296     }),
297   )
298   .subscribe({
299     next: () => {
300       this.notificationService.success('Notas', NOTES_MESSAGES.CREATE_SUCCESS);
301       void this.router.navigate(['/notes/repositories', repositoryId]);
```

Fachada: conexión con la API

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/application/facades/note.facade.ts

Fragmento real, líneas 64-70. Coordina el indicador de operación y delega la creación en NoteApi.

```
64 create(
65   repositoryId: number,
66   data: CreateNoteRequest,
67   files: File[],
68 ): Observable<ApiResponseModel<NoteModel>> {
69   return this.executeWithActionLoading(this.noteApi.create(repositoryId, data, files));
70 }
```

API Angular: datos y archivos en FormData

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/infrastructure/api/note.api.ts

Fragmento real, líneas 43-57. Agrupa título, contenido, enlaces y archivos para el POST multipart al backend.

```
43 formData.append('title', data.title);
44 formData.append('content', data.content);
45 formData.append('links', JSON.stringify(data.links));
46
47 files.forEach((file) => {
48   formData.append('files', file, file.name);
49 });
50
51 return this.post<NoteModel>(
52   this.buildUrl(
53     NOTE_API_CONFIG.endpoints.create.replace(':repositoryId', repositoryId.toString()),
54   ),
55   formData,
56 );
57 }
```

9. Frontend: presentación y edición

Fragmentos del listado, la actualización y la escritura sobre la imagen.

Tarjetas: resumen de adjuntos

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/pages/notes-page/notes-page.component.html

Fragmento real, líneas 68-75. El listado resume los archivos mediante un botón que abre la nota, en lugar de repetir una fila por adjunto.

```
68 @if (note.files.length > 0) {
69   <button type="button" class="note-attachments-button" [attr.aria-label]="Ver los ' + note.fi
   les.length + ' adjuntos de ' + note.title" (click)="openNote(note)">
70     <i class="pi pi-paperclip" aria-hidden="true"></i>
71     <span>{{ note.files.length }} {{ note.files.length === 1 ? 'archivo adjunto' : 'archivos ad
   juntos' }}</span>
72     <i class="pi pi-angle-right" aria-hidden="true"></i>
73   </button>
74 } @else {
75   <span class="note-no-files">Sin archivos adjuntos</span>
```

Los estilos de las tarjetas están en notes-page.component.scss, junto al HTML. La lógica del listado está en notes-page.component.ts, en la misma carpeta.

Detalle: envío de cambios y adjuntos conservados

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/pages/note-detail-page/note-detail-page.component.ts

Fragmento real, líneas 393-406. Envía contenido, tareas, enlaces, archivos nuevos y retainedFileIds para conservar los adjuntos existentes.

```
393 this.noteFacade.update(note.id, {
394   title: this.form.controls.title.value.trim(),
395   content: this.form.controls.content.value.trim(),
396   tasks: this.tasks(),
397   links: this.links(),
398   retainedFileIds: this.existingFiles().map((file) => file.id),
399 }, files).pipe(finally(() => this.saving.set(false))).subscribe({
400   next: (response) => {
401     if (response.data) this.setNote(response.data);
402     this.newAttachments.set([]);
403     this.newImages().forEach((image) => URL.revokeObjectURL(image.previewUrl));
404     this.newImages.set([]);
405     if (closeEditor) {
406       this.editing.set(false);
```

Editor: texto dentro del diálogo

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/components/note-image-editor/note-image-editor.component.ts

Fragmento real, líneas 280-287. Este tramo de addText coloca la entrada de IText dentro del diálogo, activa la edición y enfoca el campo para escribir.

```
280   hiddenTextareaContainer: this.editorDialog.nativeElement,
281 });
282 this.canvas.add(text);
283 this.canvas.setActiveObject(text);
284 text.enterEditing();
285 text.selectAll();
286 text.hiddenTextarea?.focus();
287 this.syncTextPanel();
```

9. Frontend: resultado y navegación

Exportación de imagen, validación de dimensiones, estado y rutas.

Editor: exportación del resultado

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/components/note-image-editor/note-image-editor.component.ts

Fragmento real, líneas 386-392. Este tramo emite un archivo PNG. El blob se genera con `canvas.toBlob` en la línea 376 del mismo archivo.

```
386 const baseName = this.file.name.replace(/\.([^/]+)$/, '');
387 this.saved.emit(
388   new File([blob], `${baseName}-editada.png`, {
389     type: 'image/png',
390     lastModified: Date.now(),
391   }),
392 );
```

Utilidades: dimensiones válidas

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/components/note-image-editor/image-editor.utils.ts

Fragmento real, líneas 1-7. La función comprueba dimensiones enteras entre 1 y 4096 píxeles. Los filtros se implementan a continuación en este archivo.

```
1 export type ImageFilter = 'grayscale' | 'sepia' | 'invert' | 'brightness' | 'contrast';
2 export const MAX_IMAGE_DIMENSION = 4096;
3
4 export function validImageDimensions(width: number, height: number): boolean {
5   return Number.isInteger(width) && Number.isInteger(height) &&
6     width >= 1 && height >= 1 && width <= MAX_IMAGE_DIMENSION && height <= MAX_IMAGE_DIMENSION;
7 }
```

Store: actualización del estado de notas

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/infrastructure/store/note.store.ts

Fragmento real, líneas 22-24. Este método actualiza el listado mediante un signal. `setNote`, en las líneas 26-28, actualiza el detalle.

```
22 setNotes(notes: PaginatedResponseModel<NoteModel> | null): void {
23   this.notesSignal.set(notes);
24 }
```

Rutas: carga de la pantalla de creación

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-frontend/src/app/features/notes/presentation/notes.routes.ts

Fragmento real, líneas 53-58. Registra `create` y carga `CreateNotePageComponent` dentro del repositorio.

```
53 path: 'create',
54 title: 'Crear Nota',
55 loadComponent: () =>
56   import('./pages/create-note-page/create-note-page.component').then(
57     (module) => module.CreateNotePageComponent,
58   ),
```

Para la estructura y los estilos del editor, revisar `note-image-editor.component.html` y `note-image-editor.component.scss` en la misma carpeta. Las pruebas de dimensiones y filtros están en `image-editor.utils.spec.ts`.

10. Backend: petición y guardado

Del endpoint HTTP a la operación de aplicación y la subida de archivos.

Controlador: creación mediante CQRS

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/presentation/controllers/notes.controller.ts

Fragmento real, líneas 71-84. Recibe multipart/form-data, aplica el límite de 5 archivos y envía CreateNoteCommand.

```
71 @Post('repository/:repositoryId')
72 @ApiConsumes('multipart/form-data')
73 @ApiBody({ type: CreateNoteRequestDto })
74 @UseInterceptors(FilesInterceptor('files', 5))
75 @HttpCode(HttpStatus.CREATED)
76 createNote(
77   @Param('repositoryId', ParseIntPipe) repositoryId: number,
78   @Body() createNoteRequestDto: CreateNoteRequestDto,
79   @UploadedFiles(NoteFilesUploadPipe) files: Express.Multer.File[],
80   @CurrentUser('sub') userId: number,
81 ) {
82   return this.commandBus.execute(
83     new CreateNoteCommand(repositoryId, createNoteRequestDto, userId, files),
84   );
85 }
```

Handler: persistencia y limpieza por fallo

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/application/commands/create-note/create-note.handler.ts

Fragmento real, líneas 84-94. Primero sube recursos; luego persiste la nota. Si falla la base de datos, intenta retirar los archivos subidos.

```
84 const uploaded = await this.mediaStorage.upload(command.files ?? [], command.repositoryId, comm
and.createdBy);
85 let created: NoteEntity;
86 try {
87   created = await this.noteRepository.createWithFiles(note, uploaded);
88 } catch (error) {
89   await this.mediaStorage.remove(uploaded.map((file) => file.storagePath));
90   throw error;
91 }
92 return {
93   message: NOTE_MESSAGES.CREATED,
94   data: NoteResponseMapper.toNoteResponse(created),
95 }
```

Servicio multimedia: ruta única y subida

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/application/services/note-media-storage.service.ts

Fragmento real, líneas 19-26. Genera una clave con UUID y envía el archivo al almacenamiento. La asociación con la nota continúa en la línea 27.

```
19 const storagePath = `repositories/${repositoryId}/notes/media/${randomUUID()}${extension}`;
20 await this.storage.uploadObject({
21   path: storagePath,
22   body: file.buffer,
23   isPublic: false,
24   contentType: file.mimetype,
25   multipart: true,
26 });
```

10. Backend: datos y dependencias

Transacciones, actualización y registro de la implementación Repository.

Repositorio Prisma: nota y adjuntos

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/infrastructure/persistence/prisma-note.repository.ts

Fragmento real, líneas 160-172. La transacción guarda la nota con noteFiles. La actividad del repositorio se actualiza después, en el mismo método. La transacción cubre MySQL; no incluye el almacenamiento externo.

```
160 return this.prisma.$transaction(async (tx) => {
161     const note = await tx.note.create({
162         data: {
163             repositoryId: entity.repositoryId,
164             title: entity.title,
165             content: entity.content,
166             tasks: entity.tasks as unknown as Prisma.InputJsonValue,
167             links: entity.links as unknown as Prisma.InputJsonValue,
168             createdBy: entity.createdBy,
169             noteFiles: { create: files },
170         },
171         include: { noteFiles: { where: { status: 1 } }, createdByUser: true },
172     });
```

Actualización de adjuntos dentro de la transacción

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/infrastructure/persistence/prisma-note.repository.ts

Fragmento real, líneas 181-191. La actualización retira lógicamente los adjuntos que no aparecen en retainedFileIds. El handler de actualización coordina esta operación.

```
181 async updateWithFiles(
182     id: number,
183     data: Partial<NoteEntity>,
184     retainedFileIds: number[],
185     files: NoteAttachment[],
186 ): Promise<NoteEntity> {
187     return this.prisma.$transaction(async (tx) => {
188         await tx.noteFile.updateMany({
189             where: { noteId: id, status: 1, id: { notIn: retainedFileIds } },
190             data: { status: 0, updatedBy: data.updatedBy },
191         });
```

Handler de actualización: D:/PR-26-PRx-main-main
(1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/application/commands/
update-note/update-note.handler.ts

Módulo: contrato e implementación

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/notes.module.ts

Fragmento real, líneas 31-38. NestJS asocia NoteRepository con PrismaNoteRepository y NoteFileRepository con su implementación. Los contratos están en domain/repositories/.

```
31 {
32     provide: NoteRepository,
33     useClass: PrismaNoteRepository,
34 },
35 {
36     provide: NoteFileRepository,
37     useClass: PrismaNoteFileRepository,
38 },
```

10. Backend: archivos y pruebas

Recuperación de recursos, esquema de notas y comprobación de fallos.

Consulta: URL y nombre del archivo

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/application/queries/get-note-file-download-url/get-note-file-download-url.handler.ts

Fragmento real, líneas 70-80. Tras validar permisos, obtiene la URL de lectura. Este extracto no demuestra por sí solo que el navegador abra correctamente un PDF.

```
70 const url = await this.tigrisStorageService.getReadUrl(  
71   noteFile.storagePath,  
72   false,  
73 );  
74  
75 return {  
76   data: {  
77     url,  
78     fileName: noteFile.name,  
79   },  
80 };
```

Implementación de almacenamiento: D:/PR-26-PRx-main-main
(1)/PR-26-PRx-main-main/prx-backend/src/shared/infrastructure/storage/
tigris-storage.service.ts

Prisma: contenido estructurado de la nota

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/prisma/schema.prisma

Fragmento real, líneas 729-737. El modelo de nota almacena contenido, tareas y enlaces. Las relaciones con adjuntos se definen en las líneas posteriores.

```
729 repositoryId Int @db.UnsignedInt  
730  
731 // Atributos requeridos  
732 title String @db.VarChar(150)  
733 content String @db.Text  
734  
735 // Atributos opcionales  
736 tasks Json?  
737 links Json?
```

Prueba con mocks: limpieza ante fallo de datos

D:/PR-26-PRx-main-main (1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/application/commands/note-media-handlers.spec.ts

Fragmento real, líneas 39-44. Simula DB_FAILED y comprueba la eliminación de las subidas. Es una prueba aislada de coordinación, no una integración real con MySQL.

```
39 it('limpia uploads cuando falla la transacción de creación', async () => {  
40   notes.createWithFiles.mockRejectedValue(new Error('DB_FAILED'));  
41   const handler = new CreateNoteHandler(notes, repos, members, media);  
42   await expect(handler.execute({ repositoryId: 5, createdBy: 2, dto: { title: 'nota', content  
43     : 'texto' }, files: [] })).rejects.toThrow('DB_FAILED');  
44   expect(media.remove).toHaveBeenCalled([ 'new-path' ]);  
45 });
```

Pruebas del servicio multimedia: D:/PR-26-PRx-main-main
(1)/PR-26-PRx-main-main/prx-backend/src/modules/notes/application/services/
note-media-storage.service.spec.ts

Recorrido de revisión: componente Angular / fachada / NoteApi / NotesController / handler / servicio multimedia y repositorio. Los extractos muestran puntos de conexión y responsabilidades; no constituyen un listado automático de cambios de Git.

11. Validación de la entrega

Comprobaciones ejecutadas el 09/10/2026 sobre dev-huamani.

Comprobación	Resultado observado
Backend	npm run build: correcto. npm test -- --runInBand: 4 suites, 21 pruebas aprobadas. El servidor compilado respondió en el puerto 3000.
Frontend	npm run build: correcto. npm test -- --watch=false: 2 archivos, 22 pruebas aprobadas. Angular conserva tres avisos de presupuesto SCSS (6,06; 7,47 y 6,85 kB).
Acceso	Registro directo devuelve sesión y usuario estándar sin generar código de correo. Login posterior correcto. Correo y usuario duplicados: 409. Contraseña incorrecta: 401.
Notas y archivos	Creación HTTP de repositorio privado y nota con PDF/PNG. Descargas idénticas al original. Actualización de tareas, enlaces y adjunto conservado persistida. Seis archivos: 400; título duplicado: 400; acceso ajeno: 403; sin token: 401. Eliminación lógica comprobada.
Alcance	Pruebas con MySQL y almacenamiento disponibles localmente. No equivalen a una certificación completa de producción o de todos los módulos y navegadores.

Ajustes adicionales de esta revisión

Se aisló la salida de compilación en dist-auth y el cliente Prisma en generated/prisma-auth. start:dev utiliza dev.cjs para evitar la limpieza de dist durante el modo watch. El seed recibe SEED_ADMIN_PASSWORD desde el entorno local. Se retiraron contraseñas de ejemplos y README. El visor cancela peticiones y libera sus blob al cerrar.

Ubicación de las comprobaciones

Backend: src/modules/auth/application/commands/register-request/register-request.handler.spec.ts;
src/modules/notes/application/commands/note-media-handlers.spec.ts;
src/modules/notes/application/services/note-media-storage.service.spec.ts;
src/shared/infrastructure/storage/tigris-storage.service.spec.ts.

Frontend: src/app/features/notes/presentation/components/note-image-editor/image-editor.utils.spec.ts y
presentation/pages/note-detail-page/note-detail-page.component.spec.ts, bajo features/notes.

Configuración local y documentación

El correo SMTP se configura exclusivamente en prx-backend/.env mediante las variables MAIL_* (MAIL_HOST, MAIL_PORT, MAIL_USER, MAIL_PASS y MAIL_FROM) de .env.example. El registro directo no necesita envío de un código; las operaciones de recuperación existentes conservan su correo. El Manual Técnico original queda intacto y excluido de GitHub por contener credenciales. Se conservan dos documentos públicos de Sprint y el respaldo privado de la documentación retirada.